

Teoría de la Computación, 2021-1

Nota de clase 3: Autómatas Finitos

Manuel Soto Romero

23 de marzo de 2021
Colegio de Ciencia y Tecnología UACM

3.1. Autómatas Finitos Deterministas

El primer tipo de máquina abstracta que estudiaremos son los Autómatas Finitos, que permiten reconocer cadenas pertenecientes a Lenguajes Regulares. Estos autómatas sólo pueden estar en un único estado después de leer cualquier secuencia de entradas. Se les llama deterministas porque para cada entrada sólo existe uno y sólo un estado al que el autómata puede moverse desde el estado actual.

Formalicemos esto un poco. Veamos la definición de Autómata Finito Determinista (AFD).

Definición 3.1. (*Autómata Finito Determinista*) Formalmente, un Autómata Finito Determinista (AFD) M es una quintupla $M = \langle Q, \Sigma, \delta, q_0, F \rangle$, donde:

- Q es un conjunto finito de estados
- Σ es el alfabeto de entrada
- q_0 es el estado inicial, con $q_0 \in Q$
- F es el conjunto de estados finales
- $\delta : Q \times \Sigma \rightarrow Q$ es la función de transición entre estados, donde $\delta(q, a) = p$ se representa en forma de diagrama o por medio de una tabla de transición.

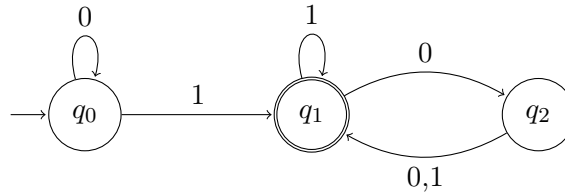
Por ejemplo, en el siguiente autómata tenemos los siguientes elementos:

- $\Sigma = \{0, 1\}$
- $Q = \{q_0, q_1, q_2\}$
- q_0 es el estado inicial.
- $F = \{q_1\}$

Para representar las transiciones, podemos dibujar el autómata como un diagrama donde:

- Cada estado es denotado por un círculo.

- Si un estado es inicial, tendrá una flecha que apunta hacia él sin tener un origen.
- Si un estado es final, se denota con un círculo doble.
- Para pasar de un estado a otro se dibujan aristas que representan con qué símbolo moverse.



En este diagrama por ejemplo, supongamos que queremos leer la cadena 011011. Para procesarla con el autómata debemos seguir la secuencia de estado marcada. Entonces:

1. Nos posicionamos en q_0 , dado que nuestro primer símbolo es 0, no nos movemos, nos quedamos en dicho estado como indica la arista del estado.
2. El siguiente símbolo es un 1 que nos lleva al estado q_1 .
3. El siguiente símbolo 1 nos deja en el estado q_1 .
4. El siguiente símbolo 0 nos lleva al estado q_2 .
5. El siguiente símbolo 1 nos lleva al estado q_1 nuevamente.
6. El siguiente símbolo 1 nos deja en el estado q_1 .
7. Dado que acabamos de procesar todos los símbolos de la cadena y terminamos en un estado final, podemos concluir que la cadena fue aceptada por el autómata.

Si no llegamos a un estado final, se dice que la cadena fue rechazada por el autómata. Revisemos ahora, la representación de estas transiciones como tabla:

		0	1
$\rightarrow$	q_0	q_0	q_1
F	q_1	q_2	q_1
	q_2	q_1	q_1

Con $\rightarrow$ señalamos el estado inicial y con **F** los estados finales.

Es importante notar que nuestra función de transición δ , sólo acepta un símbolo. Por ejemplo, para nuestro autómata anterior, tenemos:

$$\delta(q_0, 1) = q_1$$

Sin embargo, en ocasiones es útil modelar esta función de manera que nos muestre el camino completo pasando por las transiciones, es decir, que en lugar de recibir un símbolo, recibamos una cadena y nos muestre dónde termina dicha cadena. Por ejemplo:

$$\delta(q_0, 011011) = q_1$$

Llamaremos a esta nueva δ , la función de transición extendida y la denotaremos por $\hat{\delta}$. Su definición es la siguiente:

Definición 3.2. (*Función de Transición Extendida*) La función de transición extendida, $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$, se define inductivamente en términos de δ como sigue. Sea $q \in Q, a \in \Sigma$ y $x \in \Sigma^*$, entonces:

$$\begin{aligned}\hat{\delta}(q, \varepsilon) &\rightarrow q \\ \hat{\delta}(q, xa) &\rightarrow \delta(\hat{\delta}(q, x), a)\end{aligned}$$

Finalmente, decimos que el lenguaje de un AFD $M = \langle Q, \Sigma, \delta, q_0, F \rangle$ denotado por $L(M)$ se define como:

$$L(M) = \{w : \hat{\delta}(q_0, w) \in F\}$$

Es decir, son todas las cadenas que procesadas por el autómata desde el estado inicial, terminan en un estado final.

Ejemplo 3.1. Diseñar un AFD que acepte el lenguaje

$$L = \{w : w \text{ tiene un número par de ceros y un número par de unos}\}$$

La tarea de los estados de este AFD es la de contar el número de ceros y el número de unos contando en módulo 2. Es decir, el estado se emplea para recordar si el número de ceros es par o impar hasta el momento y también para recordar si el número de unos leídos hasta el momento es par o impar. Existen por lo tanto cuatro estados que pueden interpretarse de la siguiente forma:

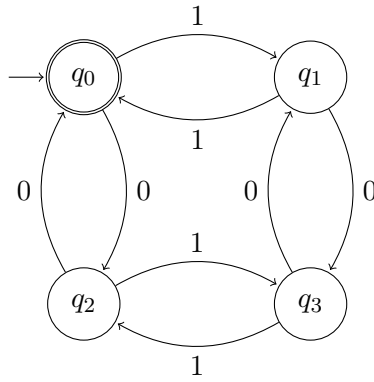
- q_0 : Tanto el número de ceros como el de unos leídos hasta el momento es par.
- q_1 : El número de ceros leídos hasta el momento es par, pero el de unos es impar.
- q_2 : El número de unos leídos hasta el momento es par, pero el de ceros es impar.
- q_3 : Tanto el número de ceros como el de unos leídos hasta el momento es impar.

El estado q_0 es por lo tanto el estado inicial así como el único estado de aceptación. Es el estado inicial porque antes de leer cualquier cosa, la cantidad de ceros y unos leídos hasta ese momento es igual a cero y cero es par. Es el único estado de aceptación porque describe de forma exacta la condición para que una secuencia de ceros y unos pertenezca al lenguaje L .

Con esto, definimos nuestro autómata con:

$$\langle \{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_0\} \rangle$$

Representamos la función de transición en formato de diagrama y de tabla.



		0	1
F →	q0	q2	q1
	q1	q3	q0
	q2	q0	q3
	q3	q1	q2

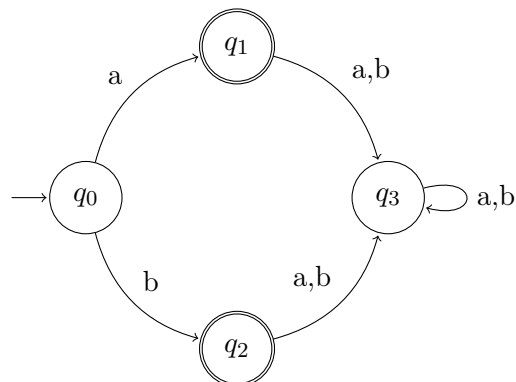
Ejercicio 3.1. Obtén los siguientes AFD:

- Un AFD que dado el alfabeto $\Sigma = \{0, 1\}$ reconozca el lenguaje de todas las cadenas que inician en 0.
- Un AFD que dado el alfabeto $\Sigma = \{0, 1\}$ reconozca el lenguaje de todas las cadenas que terminan en 1.
- Un AFD que dado el alfabeto $\Sigma = \{0, 1\}$ reconozca el lenguaje de todas las cadenas que contienen la subcadena 01.

3.1.1. Minimización de AFD

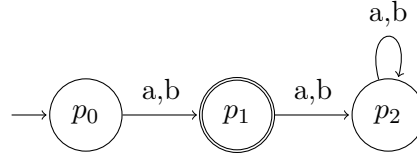
Al diseñar AFD, podemos tener distintas versiones que reconocen el mismo lenguaje. Esto en un principio, no representa ningún problema, sin embargo, podemos llegar a tener casos donde el autómata tenga estados y/o transiciones que en realidad no son del todo esenciales. Veamos algunos ejemplos:

- Analicemos por ejemplo el siguiente AFD, ¿qué lenguaje reconoce?

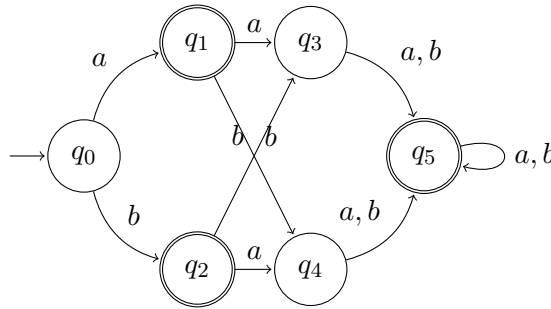


Es fácil ver que el lenguaje admitido por este autómata es $\{a, b\}^*$, sin embargo, podemos notar a partir del estado q_0 , que sin importar si leemos una a o una b , vamos a un estado final, con lo cual, parece ser que tener dos estados no es del todo necesario. De la misma forma, en ambos casos, una vez que llegamos a un estado de aceptación y leer cualquier otro símbolo del alfabeto, llegamos al estado q_3 lo cual reafirma el hecho de que no es necesario tener estado diferentes.

A partir del análisis anterior, podemos modificar el autómata como sigue:



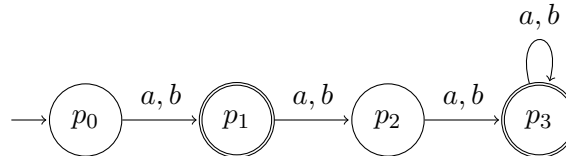
- Veamos ahora el siguiente AFD, ¿qué lenguaje reconoce?



Los estados q_1 y q_2 nos hacen notar que, al igual que el autómata anterior, el lenguaje reconocido es $\{a, b\}^*$, sin embargo, al seguir analizando las transiciones, vemos que también acepta cadenas con una longitud de al menos 3. Es decir, el lenguaje aceptado es:

$$\{a, b\}^* \cup \{w \in \{a, b\}^* : |w| \geq 3\}$$

Vemos que tanto q_1 como q_2 se pueden colapsar, de la misma forma que en el ejemplo anterior. Por otro lado, los estados q_3 y q_4 son prácticamente idénticos por lo que también podemos colapsarlos, dejando el siguiente AFD:



En ambos ejemplos hemos *colapsado* estados que tienen el mismo comportamiento, es decir, son *equivalentes*. Esta noción de equivalencia y el proceso de decisión sobre si colapsar o no un par de estados, puede realizarse algorítmicamente. Veamos la definición de equivalencia:

Definición 3.3. (Relación de equivalencia entre estados) Dado un AFD $\langle Q, \Sigma, \delta, q_0, F \rangle$ decimos que dos estados p y q están relacionados si y sólo si

$$\forall x \in \Sigma^*, \hat{\delta}(p, x) \in F \leftrightarrow \hat{\delta}(q, x) \in F$$

Denotamos esto como

$$p \approx q$$

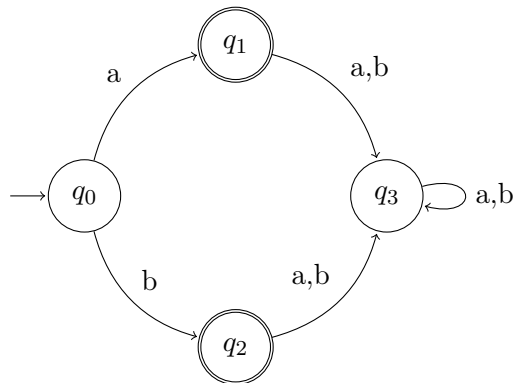
Intuitivamente, esta definición nos indica tres cosas importantes:

1. No importa qué cadena tomemos del alfabeto, si llegamos a un estado final partiendo de p o q es equivalente.
2. No necesariamente p y q deben ser iniciales, puede ser cualquier estado. tampoco deben ser finales.
3. Tienen el mismo comportamiento bajo todas las cadenas posibles.

Partiendo de esta definición, se define el *algoritmo de minimización* como sigue:

1. Escribir una tabla para cada uno de los pares de estados $\{p, q\}$, inicialmente sin marca alguna.
2. Marcar la celda correspondiente a $\{p, q\}$ si $p \in F$ y $q \notin F$ o viceversa.
3. Repetir lo siguiente hasta que no se presenten cambios: Si existe un par sin marca $\{p, q\}$ tal que $\{\delta(p, a), \delta(q, a)\}$ está marcado por alguna $a \in \Sigma$, entonces, marcamos la celda $\{p, q\}$.
4. Al terminar $p \approx q$ si y sólo si $\{p, q\}$ no tiene marca.

Veamos cómo aplicar el algoritmo con el AFD revisado anteriormente:



Construimos la tabla para cada par de estados en el autómata.

	q_0				
		q_1			
			q_2		
				q_3	
					q_4

Para cada celda, marcarla si uno de los estados es final y el otro no.

q_0			
✓	q_1		
✓		q_2	
	✓	✓	q_3

Ahora revisamos las celdas vacías y analizamos las transiciones con cada uno. Por ejemplo, analicemos $\{q_0, q_3\}$. Tenemos como alfabeto $\{a, b\}$, de forma tal que $\delta(q_0, a) = q_1$, $\delta(q_3, a) = q_3$, $\delta(q_0, b) = q_2$, $\delta(q_3, b) = q_3$, con lo cual debemos verificar si $\{q_1, q_3\}$ o $\{q_2, q_3\}$ están marcados. Si están marcados, marcamos la celda $\{q_0, q_3\}$. En este caso vemos que $\{q_1, q_3\}$ sí está marcada, por lo tanto marcamos la celda original.

q_0			
✓	q_1		
✓		q_2	
✓	✓	✓	q_3

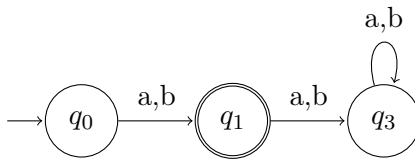
Realizando el mismo análisis vemos que no existe una celda marcada, por lo tanto dejamos dicha celda vacía y dado que el proceso se repite hasta que no haya cambios. Nuestra tabla final queda como

q_0			
✓	q_1		
✓		q_2	
✓	✓	✓	q_3

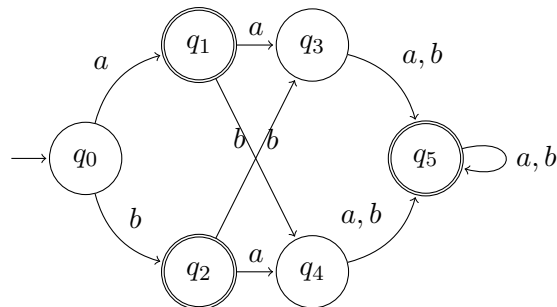
En este caso, nuestra única celda vacía fue $\{q_1, q_2\}$ con lo cual concluimos que

$$q_1 \approx q_2$$

y podemos colapsar entonces dichos estados, por ejemplo, cambiando todos las transiciones que llegaban y salían de q_2 a q_1 :



Ejercicio 3.2. Minimiza el AFD revisado anteriormente, por medio del algoritmo descrito:



3.2. Autómatas Finitos No Deterministas

Estos autómatas pueden estar en varios estados a la vez después de leer cualquier secuencia de entradas. Se les llama no deterministas pues para cada entrada sólo existe más de un estado al que el autómata puede moverse desde el estado actual.

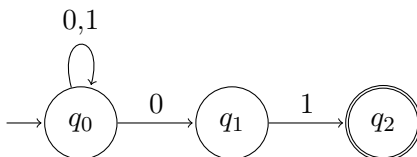
Formalicemos esto un poco. Veamos la definición de Autómata Finito No Determinista (AFN).

Definición 3.4. (*Autómata Finito No Determinista*) Formalmente, un Autómata Finito No Determinista (AFN) M es una quintupla $M = \langle Q, \Sigma, \delta, q_0, F \rangle$, donde:

- Q es un conjunto finito de estados
- Σ es el alfabeto de entrada
- q_0 es el estado inicial, con $q_0 \in Q$
- F es el conjunto de estados finales
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ es la función de transición entre estados, es decir, la salida de nuestra función de transición es un subconjunto de Q .

Por ejemplo, en el siguiente autómata tenemos los siguientes elementos:

- $\Sigma = \{0, 1\}$
- $Q = \{q_0, q_1, q_2\}$
- q_0 es el estado inicial.
- $F = \{q_2\}$



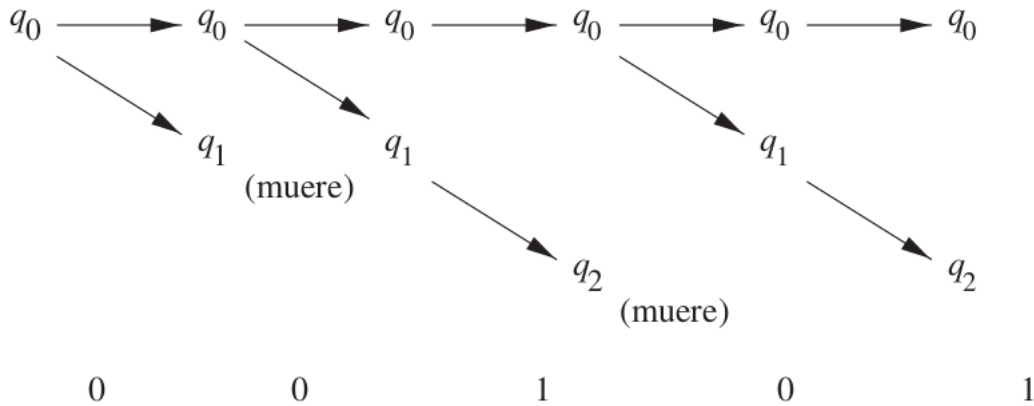
		0	1
→	q_0	$\{q_0, q_1\}$	$\{q_0\}$
	q_1	$\emptyset$	$\{q_2\}$
F	q_2	$\emptyset$	$\emptyset$

Este autómata reconoce cadenas formadas por ceros y unos que terminan en 01. Notemos que el estado inicial q_0 puede llegar a diferentes estados con el símbolo cero y que además hay algunos estados que no tienen transiciones definidas, por ejemplo q_1 con cero no tiene transición y q_2 no tiene ninguna transición. Lo cuál es válido pues $\emptyset$ es subconjunto de cualquier conjunto.

Por ejemplo, el siguiente diagrama muestra cómo el AFN procesa la entrada 00101: Parte siempre de su estado inicial q_0 . Cuando lee el primero 0, el AFN puede pasar al estado q_0 o q_1 , por lo que hace ambos. Estos dos hilos se indican en la segunda columna del diagrama.

A continuación, lee el segundo 0. El estado q_0 puede pasar de nuevo q_0 o a q_1 . Sin embargo, el estado q_1 no tiene transición para la entrada 0, por lo que el autómata se detiene. Cuando se produce la tercera entrada, un 1, hay que considerar las transiciones tanto de q_0 como de q_1 . Comprobamos que q_0 sólo pasa a q_0 cuando la entrada es 1, mientras que el estado q_1 pasa sólo al estado q_2 . Por tanto después de leer la secuencia, el AFN se encuentra en los estados q_0 y q_2 . Dado que q_2 es un estado final, el AFN acepta la secuencia 001.

Sin embargo, la entrada no ha terminado. La cuarta entrada, un 0, hace el hilo de q_2 muera, mientras que q_0 va tanto a q_0 como a q_1 . La última entrada, un 1, hace que q_0 permanezca en este mismo estado y q_1 pasa al estado q_2 . Como de nuevo estamos en un estado de aceptación, la secuencia 00101 es aceptada.



De manera general, diremos que una cadena es *aceptada* por un AFN si existe al menos una trayectoria de ejecución que termina en un estado final y diremos que es *rechazada* cuando todas las posibles trayectorias de ejecución se estancan (mueren) o terminan en un estado que no es final. Finalmente, mostramos la función de transición extendida $\hat{\delta}$ como sigue:

Definición 3.5. (*Función de Transición Extendida*) La función de transición extendida, $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$, se define inductivamente en términos de δ como sigue. Sea $q \in Q$, $a \in \Sigma$ y $x \in \Sigma^*$, entonces:

$$\begin{aligned} \hat{\delta}(q, \varepsilon) &\rightarrow \{q\} \\ \hat{\delta}(q, xa) &\rightarrow \bigcup_{p_i \in \hat{\delta}(q, x)} \delta(p_i, a) \end{aligned}$$

Por ejemplo, podemos usar $\hat{\delta}$ para describir el procesamiento de la entrada 00101 por el AFN de nuestro ejemplo anterior.

1. $\hat{\delta}(q_0, \varepsilon) = \{q_0\}$
2. $\hat{\delta}(q_0, 0) = \delta(q_0, 0) = \{q_0, q_1\}$

3. $\hat{\delta}(q_0, 00) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$
4. $\hat{\delta}(q_0, 001) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$
5. $\hat{\delta}(q_0, 0010) = \delta(q_0, 0) \cup \delta(q_2, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$
6. $\hat{\delta}(q_0, 00101) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$

Finalmente, decimos que el lenguaje de un AFN $N = \langle Q, \Sigma, \delta, q_0, F \rangle$ denotado por $L(N)$ se define como:

$$L(N) = \left\{ w \in \Sigma^* : \hat{\delta}(q_0, w) \cap F \neq \emptyset \right\}$$

Es decir, son todas las cadenas que procesadas por el autómata desde el estado inicial, llegan por alguna trayectoria a algún estado final.

Ejercicio 3.3. Obtén los siguientes AFN:

- Un AFN que dado el alfabeto $\Sigma = \{0, 1\}$ reconozca el lenguaje de todas las cadenas que inician en 0.
- Un AFN que dado el alfabeto $\Sigma = \{0, 1\}$ reconozca el lenguaje de todas las cadenas que terminan en 1.
- Un AFN que dado el alfabeto $\Sigma = \{0, 1\}$ reconozca el lenguaje de todas las cadenas que contienen la subcadena 01.

3.3. Equivalencia entre AFDs y AFNs

El poder de un autómata se mide en términos de la habilidad para aceptar lenguajes. Observemos que un AFD es un caso especial de un AFN, de modo que todo lo que un AFD puede hacer, un AFN también lo puede hacer, de hecho para cada AFN existe un AFD que acepta el mismo lenguaje. Además, como seguramente te habrás dado cuenta, es más sencillo construir, especificar y comprender un AFN debido a que son más concisos. En general, los AFNs pueden ser usados para capturar patrones en procesadores de texto, pero optamos por los AFD cuando deseamos encontrar dichos patrones.

Teorema 3.1. *Para cada AFN N , existe un AFD M con $L(N) = L(M)$.*

La demostración de este teorema queda fuera de los alcances de este curso, por lo que en su lugar, presentaremos el proceso para transformar un AFN en un AFD por medio de la llamada *construcción por subconjuntos*:

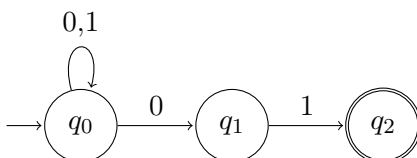
1. Construir una tabla de transiciones a partir del conjunto potencia, que muestre los estados a donde se puede llegar con cada estado de dichos conjuntos. Al conjunto que tenga como único elemento al estado inicial, lo marcamos como final y a todos los conjunto que contengan al estado final original, los marcamos también como finales.
2. Eliminar estados que no sean alcanzables.

3. Renombrar los estados.

Recordemos que el conjunto potencia se define como el conjunto formado por todos los subconjuntos de un conjunto dado. Por ejemplo, dado el conjunto $A = \{1, 2, 3\}$, entonces

$$\mathcal{P}(A) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$$

Veamos entonces cómo aplicar este algoritmo con el autómata revisado anteriormente:



1. Construimos la tabla de transiciones a partir del conjunto potencia

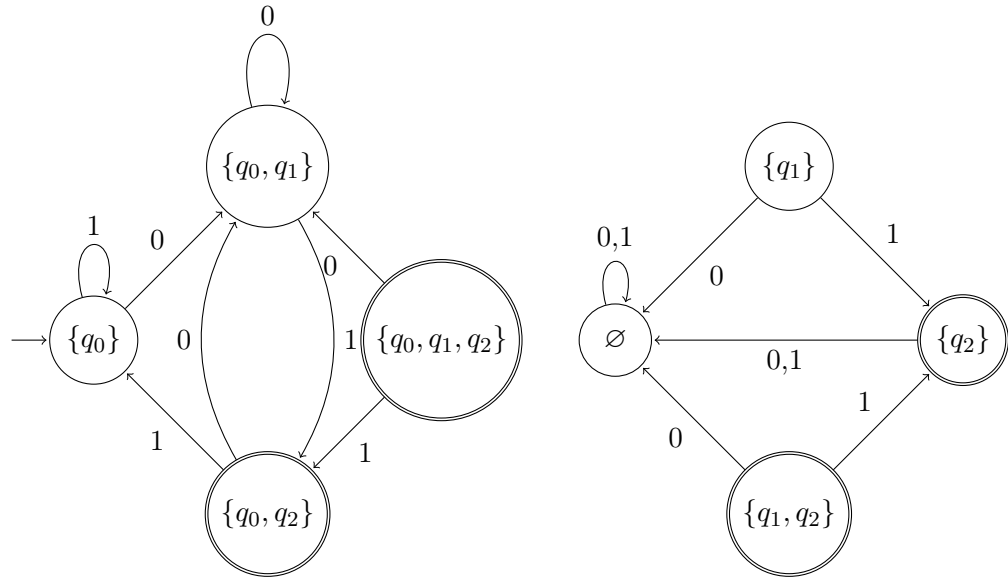
■ Inicialización:

		0	1
	$\emptyset$		
$\rightarrow$	$\{q_0\}$		
	$\{q_1\}$		
F	$\{q_2\}$		
	$\{q_0, q_1\}$		
F	$\{q_0, q_2\}$		
F	$\{q_1, q_2\}$		
F	$\{q_0, q_1, q_2\}$		

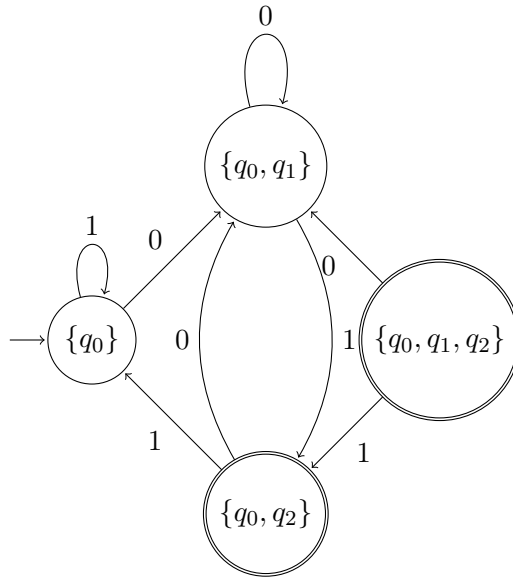
■ Llenamos la tabla con los resultados de transitar desde cada uno de los estados.

		0	1
	$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow$	$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
	$\{q_1\}$	$\emptyset$	$\{q_2\}$
F	$\{q_2\}$	$\emptyset$	$\emptyset$
	$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
F	$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
F	$\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
F	$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

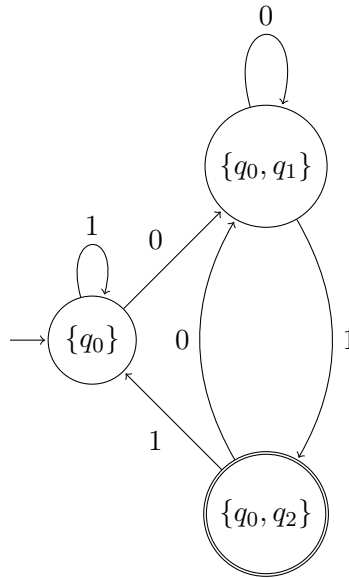
■ Construimos el autómata resultante



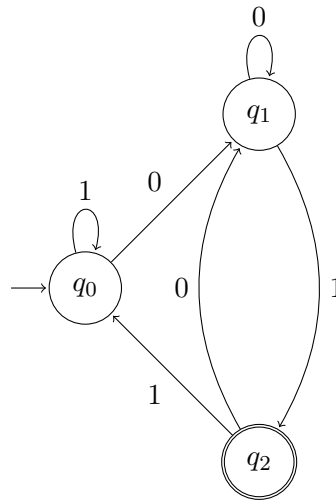
- Notamos que nuestro autómata se encuentra *disconexo*, esto ocurre debido a que tenemos estados inalcanzables. Partiendo desde el estado inicial detectamos estos estados. Claramente el subautómata de la derecha es inalcanzable, por lo tanto lo eliminamos.



De la misma forma, el estado $\{q_0, q_1, q_2\}$ es inalcanzable pues no hay ningún estado que llegue a él. Lo eliminamos también.

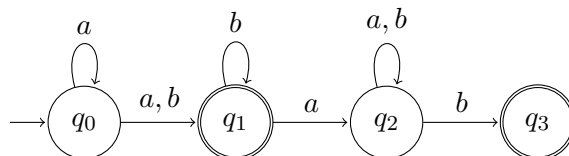


- Finalmente, renombramos los estados.



Notamos que el autómata resultante es un AFD y que además reconoce el mismo lenguaje: cadenas que terminen con 01.

Ejercicio 3.4. Mediante construcción por subconjuntos, transforma el siguiente AFN a AFD. ¿Qué lenguaje reconoce?



3.4. Equivalencia entre Autómatas Finitos y Gramáticas Regulares

Al igual que entre los AFDs y AFNs existe una equivalencia, también existe una equivalencia entre las gramáticas. El procedimiento de transformación es bastante simple:

- Se deben asociar a los símbolos terminales de la gramática (las variables) los estados de un autómata. Así, para cada regla $A \rightarrow bC$ en la gramática tenemos una transición $\delta(A, b) = C$. Observemos que las variables se comportan como estados.
- Para las reglas de constantes como $A \rightarrow b$ se generan transiciones $\delta(A, b) = Z$ donde Z es un nuevo estado para el que no hay un símbolo terminal asociado; Z es un estado final del autómata.

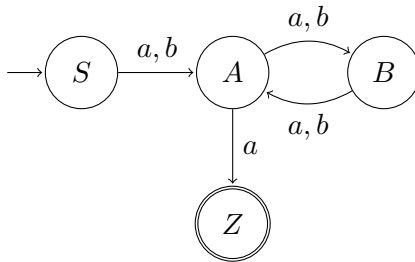
Por ejemplo, supongamos la siguiente gramática:

$$\begin{aligned}
 S &\rightarrow aA \\
 S &\rightarrow bA \\
 A &\rightarrow aB \\
 A &\rightarrow bB \\
 A &\rightarrow a \\
 B &\rightarrow aA \\
 B &\rightarrow bA
 \end{aligned}$$

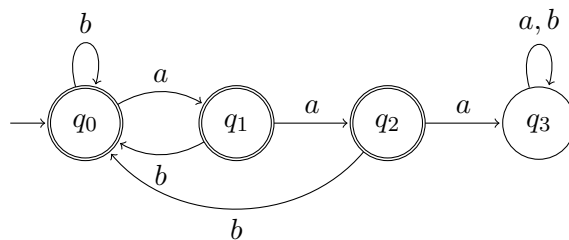
Tenemos las siguientes transiciones:

$$\begin{aligned}
 \delta(S, a) &= A \\
 \delta(S, b) &= A \\
 \delta(A, a) &= B \\
 \delta(A, b) &= B \\
 \delta(A, a) &= Z \\
 \delta(B, a) &= A \\
 \delta(B, b) &= A
 \end{aligned}$$

Quedando el siguiente diagrama:



El proceso inverso es similar. Basta con revisar las transiciones de manera que si tenemos una transición $\delta(q_0, a) = q_1$, entonces tenemos una regla de producción $Q_0 \rightarrow aQ_1$ y además si tenemos una transición $\delta(q_1, a) = q_2$ con $q_2 \in F$, entonces tenemos una regla $Q_1 \rightarrow a$. Por ejemplo, la gramática por el siguiente autómata:



tenemos las siguientes reglas, obtenidas a partir de las transiciones:

$$\begin{array}{llll}
\delta(q_0, a) & = & q_1 & Q_0 \rightarrow aQ_1 \\
\delta(q_0, b) & = & q_0 & Q_0 \rightarrow bQ_0 \\
\delta(q_0, a) & = & q_1 & Q_0 \rightarrow a \\
\delta(q_0, b) & = & q_0 & Q_0 \rightarrow b \\
\delta(q_1, a) & = & q_2 & Q_1 \rightarrow aQ_2 \\
\delta(q_1, b) & = & q_0 & Q_1 \rightarrow bQ_0 \\
\delta(q_1, a) & = & q_2 & Q_1 \rightarrow a \\
\delta(q_1, b) & = & q_0 & Q_1 \rightarrow b \\
\delta(q_2, a) & = & q_3 & Q_2 \rightarrow aQ_3 \\
\delta(q_2, b) & = & q_0 & Q_2 \rightarrow bQ_0 \\
\delta(q_2, b) & = & q_0 & Q_2 \rightarrow b \\
\delta(q_3, a) & = & q_3 & Q_3 \rightarrow aQ_3 \\
\delta(q_3, b) & = & q_3 & Q_3 \rightarrow bQ_3
\end{array}$$

Ejercicio 3.5. Construye un AFD que acepte cadenas que no tienen tres a 's seguidas. Partiendo de este autómata proponer una gramática regular.

Ejercicio 3.6. Propón un autómata finito que reconozca el lenguaje generado por la siguiente gramática:

$$\begin{array}{ll}
A & \rightarrow aA \\
A & \rightarrow bB \\
B & \rightarrow aA \\
B & \rightarrow bC \\
B & \rightarrow b \\
C & \rightarrow aC \\
C & \rightarrow bC \\
C & \rightarrow a \\
C & \rightarrow b
\end{array}$$

Referencias

- [1] Hopcroft, J. E; Montwani, R., and Ullman, J. *Introduction to Automata Theory Language, and Computation*. Addison Wesley, third edition 2007.
- [2] Noé S. Hernández, *Notas de Clase para el curso de Autómatas y Lenguajes Formales*, Facultad de Ciencias UNAM, 2020.